Executions · Jomana

GIU Curriculum and Academic Innovation Management Platform (GCAIMP) · Milestone 2 · executions divided between the team, same split as Procedures.pdf

From the Milestone 2 document (GIU, Winter 2026, submission 15/11/2026, 11:59 pm):

d) **Executions.sql**: an example execution of every stored procedure and function (**EXEC** for procedures, **SELECT** for functions), using data from your insertions.

• The four SQL files must run in this order without errors: Tables.sql, Insertions.sql, Procedures.sql, Executions.sql.

• It is the students' responsibility to make sure that every stored procedure and function executes correctly and returns the expected output.

• Your procedures and functions must handle any errors or exceptions that may happen (e.g., invalid input, records that do not exist, duplicate records, or violated constraints) and show a clear message instead of failing.

How we do it: write your calls in `executions/<YourName>.sql` on your branch, one execution per procedure or function below, in the same order. Use data from Insertions.sql. Add a second call that triggers an error case where it makes sense. Merge into main and the CI glues it into Executions.sql.

Jomana 23 executions (one per procedure / function)

1 As a System Admin, I should be able to:

16. Add a new academic semester.
    Signature:
    Name: AddSemester.
    Type: Stored procedure.
    Input: @Name varchar(50), @AcademicYear varchar(50), @StartDate date, @EndDate date.
    Output: Confirmation message or new @SemesterID int.

18. Create a course offering (section) in a semester and assign its instructor.
    Signature:
    Name: CreateCourseOffering.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @DeliveryMode varchar(50), @EnrolledCount int, @InstructorID int.
    Output: Confirmation message.

19. Assign a teaching assistant to support a course offering.
    Signature:
    Name: AssignTAToOffering.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @TAID int.
    Output: Confirmation message.

2 As an Academic Instructor, I should be able to:

1. View the course offerings I teach in a given semester.
   Signature:

Name: ViewMyCourseOfferings.
Type: Function.
Input: @InstructorID int, @SemesterID int.
Output: Table of course offerings with course title, section, delivery mode and enrolled count.

5. Record the result of assessing an ILO in one of my course offerings (whether the target was met is calculated automatically).
   Signature:
   Name: RecordOutcomeAssessment.
   Type: Stored procedure.
   Input: @CourseCode varchar(20), @ILOCode varchar(20), @SemesterID int, @SectionID varchar(20), @MappingID int, @Instrument varchar(150), @Description varchar(1000), @TargetThreshold decimal(10,2), @AttainedValue decimal(10,2), @SampleSize int, @AnalysisNotes varchar(1000).
   Output: Confirmation message showing whether the target was met.
6. Add a corrective action to an assessment whose target was not met.
   Signature:
   Name: AddCorrectiveAction.
   Type: Stored procedure.
   Input: @AssessmentID int, @CorrectiveAction varchar(1000).
   Output: Confirmation message.
7. View the outcome assessments in my offerings that did not meet their target.
   Signature:
   Name: ViewUnmetAssessments.
   Type: Function.
   Input: @InstructorID int.
   Output: Table of assessments with ILO, offering, target, attained value and corrective action.

3 As a Teaching Assistant, I should be able to:

1. View the course offerings I support.
   Signature:
   Name: ViewSupportedOfferings.
   Type: Function.
   Input: @TAID int.
   Output: Table of course offerings with course title, semester, section and instructor name.
4. View the outcome assessments recorded for a course offering.
   Signature:
   Name: ViewOfferingAssessments.
   Type: Function.
   Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20).
   Output: Table of assessments with instrument, target, attained value and result.
5. Update the enrolled student count of an offering I support.
   Signature:
   Name: UpdateEnrolledCount.
   Type: Stored procedure.
   Input: @CourseCode varchar(20), @SemesterID int, @SectionID varchar(20), @EnrolledCount int.
   Output: Confirmation message.

4 As a Curriculum Reviewer, I should be able to:

5. Calculate the attainment rate of a PLO in a semester.
   Signature:
   Name: GetPLOAttainment.
   Type: Function.
   Input: @ProgramID int, @PLOCode varchar(20), @SemesterID int.
   Output: Percentage of linked assessments that met their target decimal(5,2).

6. View all assessments in a program that fell below target in a semester.
   Signature:
   Name: ViewAssessmentsBelowTarget.
   Type: Function.
   Input: @ProgramID int, @SemesterID int.
   Output: Table of assessments with course, ILO, target, attained value and corrective action.

7. Track whether corrective actions on an ILO improved its attainment in later semesters.
   Signature:
   Name: TrackCorrectiveActionEffect.
   Type: Function.
   Input: @CourseCode varchar(20), @ILOCode varchar(20).
   Output: Table showing semester, target, attained value, result and corrective action, ordered by semester.

11. Open a new accreditation review for a program.
    Signature:
    Name: CreateAccreditationReview.
    Type: Stored procedure.
    Input: @ProgramID int, @AccreditingBody varchar(150), @ReviewType varchar(50), @ReviewCycle varchar(50), @NextReviewDue date.
    Output: Confirmation message or new @ReviewID int.

12. Attach all approved outcome mappings of the program to an accreditation review.
    Signature:
    Name: AttachMappingsToReview.
    Type: Stored procedure.
    Input: @ReviewID int.
    Output: Confirmation message showing the number of attached mappings.

13. Attach an outcome assessment to an accreditation review.
    Signature:
    Name: AttachAssessmentToReview.
    Type: Stored procedure.
    Input: @ReviewID int, @AssessmentID int.
    Output: Confirmation message.

14. Attach a piece of initiative evidence to an accreditation review.
    Signature:
    Name: AttachEvidenceToReview.
    Type: Stored procedure.
    Input: @ReviewID int, @EvidenceID int.
    Output: Confirmation message.

15. Update the status and decision of an accreditation review.
    Signature:
    Name: UpdateAccreditationStatus.

Type: Stored procedure.
Input: @ReviewID int, @Status varchar(50), @Decision varchar(50), @SubmissionDate date.
Output: Confirmation message.

16. Generate the full evidence pack of an accreditation review.
    Signature:
    Name: GenerateAccreditationPack.
    Type: Function.
    Input: @ReviewID int.
    Output: Table listing every attached mapping, assessment and evidence item with its details.

17. View accreditation reviews due within a number of days.
    Signature:
    Name: GetUpcomingAccreditations.
    Type: Function.
    Input: @DaysAhead int.
    Output: Table showing program, accrediting body, status and next-review due date.

6 As an University Management member, I should be able to:

7. View the latest accreditation status of every program.
   Signature:
   Name: GetAccreditationOverview.
   Type: Function.
   Input: Nothing.
   Output: Table showing program, faculty, latest review status, decision and next-review due date.

8. View the outcome attainment trend of a program across semesters.
   Signature:
   Name: GetAttainmentTrend.
   Type: Function.
   Input: @ProgramID int.
   Output: Table showing semester and percentage of assessments that met their target.

7 As an Academic Member, I should be able to:

14. Be notified of upcoming accreditation deadlines for my program.
    Signature:
    Name: NotifyAccreditationDeadline.
    Type: Stored procedure.
    Input: @ProgramID int, @DaysAhead int.
    Output: Notification message.